딥러닝을 활용한 차세대 번역기

협성대학교
컴퓨터공학과
강준호, 남궁재우, 신우섭, 오준석

딥러닝을 활용한 차세대 번역기

본 논문을 학사학위 논문으로 제출합니다.

2018년 11월 8일

협성대학교
컴퓨터공학과
강준호, 남궁재우, 신우섭, 오준석

지도 교수 : 노흥식 교수님

목차

목차 ····· I
그림 차례 ····· II
국문 요약 ····· IV

1. 서론 ····· 1
    1.1 프로그램 개요 ····· 1
    1.2 개발 동기 ····· 3

2. 본론 ····· 4
    2.1 인공지능 구현 ····· 4
        2.1.1 머신러닝 ····· 4
        2.1.2 딥러닝 ····· 10
        2.1.3 Tensorflow ····· 14
        2.1.4 Python과 Tensorflow를 활용한 인공지능 구현 ····· 15
    2.2 프로그램 연동 ····· 20
        2.2.1 소켓 통신 ····· 20
        2.2.2 프로토콜 ····· 21
        2.2.3 서버 ····· 22
        2.2.4 클라이언트 ····· 25
        2.2.5 웹 ····· 27
    2.3 UI 구현 ····· 30
        2.3.1 개발 도구 ····· 30
        2.3.2 Visual Studio ····· 31
        2.3.3 안드로이드 ····· 32

3. 결론 ····· 33
    3.1 활용 사례 ····· 33
    3.2 후기 ····· 37

그림 차례

그림 1. 딥러닝 등장 시기 ·····1
그림 2. 인공지능 예시 ·····2
그림 3. Seeing AI ·····2
그림 4. 물체 인식에 대한 예 ·····4
그림 5. 머신러닝 단계 ·····5
그림 6. 트레이닝 세트의 예시 ·····6
그림 7. 머신러닝이 담겨 있는 영화 '채피' ·····7
그림 8. 지도 학습 예시 (영화 채피)·····8
그림 9. 비지도 학습 예시 (영화 채피)·····8
그림 10. 회귀분석, 분류, 클러스터링 표현도·····9
그림 11. 인공지능 기술의 집합·····10
그림 12. 뉴런과 인공 신경의 구조 ·····10
그림 13. 인공 신경망 구조 ·····11
그림 14. 오차 역전파 표현도-1 ·····12
그림 15. 오차 역전파 표현도-2 ·····12
그림 16. MNIST 데이터 그림 예시 ·····15
그림 17. MNIST 데이터 코드 ·····15
그림 18. MNIST 데이터 확인 ·····16
그림 19. 2차원 행렬로 변환하는 코드와 결과·····16
그림 20. MNIST train 코드·····17
그림 21. MNIST train 표현도·····17
그림 22. Training 데이터 테스트 코드 ·····18
그림 23. MNIST 테스트 코드와 결과 ·····19
그림 24. MNIST 최종 결과 ·····19
그림 25. 이미지를 보내고 받는 패킷 ·····21
그림 26. 이미지의 이진코드 데이터 변환 ·····21
그림 27. 분석 결과(문장)를 보내고 받는 패킷 ·····22
그림 28. 서버 구현 알고리즘 ·····22
그림 29. 소켓의 초기화부터 클라이언트 연결까지의 Python 코드 ·····22
그림 30. 소켓으로부터 읽은 데이터(값)를 대입 ·····23
그림 31. 사진 속 결과를 클라이언트로 전송 ·····23
그림 32. 사진 속 결과를 클라이언트로 전송 ·····24
그림 33. C#에서 Python 서버로 소켓을 연결하는 모습 ·····25
그림 34. 데이터를 Python 서버로 전송하고, 결과를 받아오는 모습 ·····26
그림 35. 영역 처리 알고리즘 ·····27

그림 36. PHP에서 MySQL에 연결하고 새로운 사진 정보를 삽입하는 모습 ·····28
그림 37. 저장 코드 ·····28
그림 38. PHP에서 업로드된 사진 정보를 Python이 불러오는 모습 ·····29
그림 39. Python이 번역 결과를 데이터베이스에 기록하는 모습 ·····29
그림 40. Winform으로 제작한 차세대 번역기·····31
그림 41. 덴스캡(DenseCap)의 사진 해석 결과·····34
그림 42. 구글의 사진 속 장면 인식 및 문장 묘사 기술 ·····34
그림 43. 의료기술에 적용된 딥러닝 ·····35
그림 44. 콘텐츠 업계에 적용된 딥러닝 ·····35
그림 45. 자동차 기술에 적용된 딥러닝 ·····36
그림 46. 사진을 유명 화가의 화풍으로 변환한 예 ·····36

국문 요약

딥러닝을 활용한 차세대 번역기

색인 : A.I, Deep learning, Tensorflow

4차 산업혁명의 시대가 시작된 가운데, 핵심기술인 인공지능 기술의 중요성이 강조되고 있다. 매 해를 거듭할수록 인공지능은 우리의 일상생활에 계속해서 가까워지고 있다. 하지만, 인공지능 기술이 아직 한정된 분야에서만 강세를 보이고 있다. 다양한 시도를 통하여, 앞으로 다양한 분야에서 활용되어 우리의 삶의 질 향상에 기여할 수 있을 것이라고 생각을 하였다.

인공지능 기술을 통해 현재 시도되지 않은 분야에 활용하여 실용성 있는 소프트웨어를 개발하여 사용자의 입장에서 편의를 느낄만한 프로그램을 개발하기로 결정하였다. 다양한 후보들을 물색하던 중, 네이버에서 제작한 번역기인 'PAPAGO'를 업그레이드 하여 차세대 번역기를 개발하기로 하였다.

우리나라의 번역기는 현재, 텍스트 번역기능, 음성 번역기능, 대화 번역기능, 사전 기능 등 다양한 기능들을 제공한다. 현재 제공하고 있는 기능들 또한, 번역기로서 가져야할 주요기능을 제공하고, 이외에 부수적인 기능을 편의성 있게 제공한다. 이미지 번역 기능은 텍스트로 입력이 어려울 때 유용하다. 카메라로 사진을 찍어서 손가락으로 드래그하면 그에 대한 내용이 실시간으로 번역이 된다. 하지만, 이미지 자체의 상황 또는 대상을 텍스트로 번역해주는 기능은 제공하지 않는다. 우리는 기존의 번역기에 인공지능 기술을 적용하여, 이미지를 텍스트로 번역할 수 있는 기능을 추가한 차세대 번역기를 기획하였다.

1. 서론

1.1 프로그램 개요

전 세계에서 인공지능이 화두가 되고 있고, 세계 곳곳에서 인공지능이 가지고 올 사회에 많은 관심을 가지고 있다. 이러한 것을 두고 4차 산업혁명이라고 한다. 인공지능을 이야기할 때 4차 산업혁명이 빠지지 않는 것처럼 인공지능이 사회 진화 방향을 크게 바꿀 것이라고 많은 사람들이 예측하고 있다.

인공지능 산업이 빠른 속도로 발전하고 있고, 인공지능 기술들로 인하여 세상은 빠르게 변하고 있다. 일부 분야에만 국한되어 있기보다, 다양한 방면에서 인공지능 기술을 적용하여 활발히 하여야 한다.

딥러닝으로 인한 인공지능의 발전은 인지, 학습, 추론, 행동과 같은 인간 지능 영역의 전 과정에 걸쳐 혁신적인 진화를 만들어 내고 있다. 특히 시각, 청각과 같은 감각기관에 해당하는 인지지능은 2012년을 기점으로 본격적으로 발전하였으며, 시각 인지 분야의 지능은 인간 수준을 초월하는 수준으로 구현되고 있다.

유튜브에서 발견한 2012년 영상 속의 고양이를 스스로 구분해 낸 구글의 인공지능과 이미지 속의 사물 인식 정확도를 혁신적으로 향상시킨 토론토 대학의 인공지능이 구현된 지 약 3년 만에 인간의 수준을 뛰어넘은 것이다.

매년 다양한 연구기관이 참여해 이미지 내 사물 인식의 정확도를 경쟁하는 ImageNet 경진대회 에서는 2015년 Microsoft가 96.43%의 정확도를 달성하며 인간의 인식률(94.90%)을 초월하였다. ('17년 정확도:97.85%)

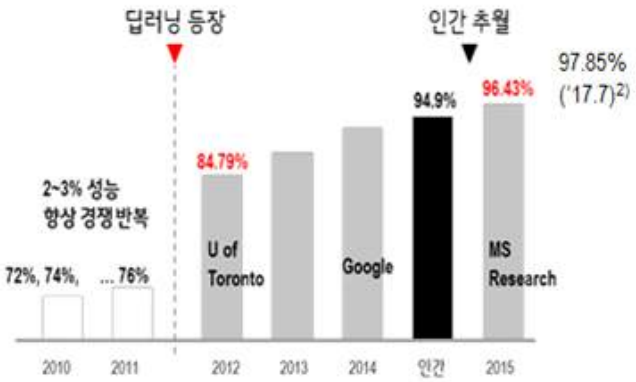

그림 1. 딥러닝 등장 시기

인공지능은 단순히 이미지 속의 사물의 종류를 인식하는 것을 뛰어넘어, 이해하기 시작했다.

사람 얼굴의 사진을 보면 남성, 여성 등과 같은 외형적 특성을 인식하는 것은 물론, 눈, 코, 입의 상관관계를 분석해 표정을 인식하거나, 구글의 2015년 논문에는 이미지 속 상황을 정확히 이해해 인간의 언어로 표현하기도 한다. 시각 지능을 통해 이미지를 인식하고 이해하게 된 인공지능은 이미지 속 상황에 대한 인간의 물음에 정확히 답을 내리기도 한다.

그림 2. 인공지능 예시

이처럼 진화하고 있는 시각 지능을 기반으로 한 인공지능은 연구 단계를 넘어서 실제 생활에 적용되며 다양한 혁신을 만들어 갈 것으로 전망된다. Microsoft는 고도화된 시각 지능을 활용한 'Seeing AI' 라는 시각 장애인용 인공지능을 발표했다. 앞을 볼 수 없는 시각 장애인의 시각을 인공지능이 대신하는 것이다. 시각 장애인에게 눈앞의 상황을 인간의 언어로 설명해 주거나, 앞에 앉아 있는 상대방의 성별, 나이, 표정 등의 정보를 제공해준다. 즉, 시각 장애를 갖는 사람들의 시각 인지 기능을 인공지능이 대신 제공해 장애인들의 일상생활을 혁신적으로 변화시킬 수 있을 것으로 전망된다.

그림 3. Seeing AI

1.2 개발 동기

알파고의 등장으로 요원할 것만 같았던 인공지능의 시대가 한 발 더 가까워졌다. 인공지능은 바둑, 안면 인식 등 많은 분야에서 사람의 능력을 뛰어넘고 있다. 최근에는 극본이나 작곡 등 창작의 영역에도 인공지능의 참여가 이어지고 있다. 최근 인공지능의 놀라운 성취는 바로 딥러닝 기술의 등장 때문이다.

앞서 설명 드린 딥러닝의 발달을 통해 인지분야기술은 인간 수준을 초월하는 수준으로 구현되고 있다. 4차 산업혁명의 주요기술로 인공지능이 언급된 가운데, 인공지능의 활용 방안을 적극적으로 연구해야 한다. 오늘날 사물인터넷은 센서와 네트워크를 활용, 각종 빅 데이터를 쏟아낸다. 인공지능은 이를 효과적으로 관리, 분석하는 소프트웨어라고 할 수 있다.

새로운 소프트웨어 개발을 통해 인공지능 기술이 추상적인 개념을 넘어, 다양한 소프트웨어와의 연계를 통해 상용화될 가능성이 있다. 아이디어와 인공지능이 결합한다면 다양한 이들의 삶의 질을 높이고, 불편을 해소할 수 있다.

차세대 번역기는 딥러닝을 통한, 시각인지 기술의 다양한 응용을 통하여 인간의 삶에 도움이 될 수 있는 기능을 주고자 기획되었다. 차세대 번역기는 기존의 번역기의 기능을 초월하여, 문자가 아닌, 이미지를 문장으로 번역할 수 있다. 차세대 번역기의 기술은 이미지를 다양한 언어의 문장으로 번역할 수 있고, 또한 문장으로 만들기 때문에, 유아의 영어교육 프로그램을 보조하여 추후 사교육비 절감의 효과, 시각장애인의 시각적 이미지를 문장으로 변환하고 이를 소리로 출력하여, 시각적인 불편함을 가지고 있는 사람들에게도 편리하게 사용될 수 있다.

이처럼 차세대 번역기는 인공지능과 번역기의 새로운 연계를 통하여 많은 사람들의 불편함을 해소하고 삶의 질 향상 등, 다양한 이점을 가지고 있다. 인공지능에 대한 많은 사람들의 관심과, 개발을 위한 적극적인 노력이 이루어진다면, 인공지능을 인간의 삶에 적용하여 더욱 편리한 시대가 올 것 이라고 생각하여 차세대 번역기를 만들게 되었다.

2. 본론

2.1 인공지능 구현

2.1.1 머신러닝

머신러닝은 프로그래머의 프로그램 없이 컴퓨터에게 배우는 능력을 갖게 하는 컴퓨터 과학의 한 종류이다. 사람이 직접 손으로 하는 코딩 없이, 수집한 수많은 데이터를 통해 주어진 입력(input)에 대해 올바른 출력(output)을 하게 하는 것이다.

가령, 어질러져 있는 장면에서 조명 조건이 바뀌었을 때 특이한 방향에서 3차원 물체를 인식하도록 하는 프로그램이 있을 것이다. 사람에게는 아주 간단한 일이다. 빛의 밝기와 빛이 들어오는 방향에 따라서 물체의 색과 그림자의 모양이 달라져도 위, 옆, 아래, 대각선의 보는 방향을 어느 쪽으로 바꾸더라도 '책상'을 '책상'이라고 인식할 수 있다.

그림 4. 물체 인식에 대한 예

하지만 우리는 우리의 뇌에서 정확히 어떤 일이 일어나서 이러한 작업을 수행하는지 알지 못한다. 설령 알고 있다고 해도 그 과정을 프로그램으로 작성하는 과정은 무지막지하게 복잡해질 것이다. 또 하나의 예는, 신용 카드 거래가 사기일 확률을 계산하는 프로그램 작성이다. 사기라고 확신할 수 있는 간단하면서도 신뢰할 수 있는 강력한 규칙은 아마도 없을 것이다. 그렇기에 많은 수의 약한 규칙들을 결합할 필요가 있다. 사기의 표적은 계속 이동하기 때문에 프로그램은 상황에 맞게 계속 변화해야 한다.

위에서 언급한 문제들처럼 인간의 프로그래밍만으로는 해결하기 힘든 문제들이 많이 있다. 따라서 특정한 작업을 수행하는 프로그램을 사람이 작성하는 대신 기계가 프로그램

을 생산하는 방법으로 접근한 것이다.

즉, 수많은 예제 데이터를 머신 러닝 알고리즘이 습득하게 하여 프로그램을 생산하는 것이다. 만약 그 과정이 올바르게 되었다면, 만들어진 프로그램은 훈련시킨 것처럼 새로운 경우에 대해서도 작동할 것이다. 더불어 기술의 발전으로 방대한 양의 계산을 수행하는 컴퓨터의 가격이 저렴해졌기 때문에 프로그래머를 고용하여 복잡한 프로그래밍을 하는 것보다 머신 러닝을 통하여 프로그램을 생산하는 것이 시간과 비용 면에서 효율적이라는 이유도 있다.

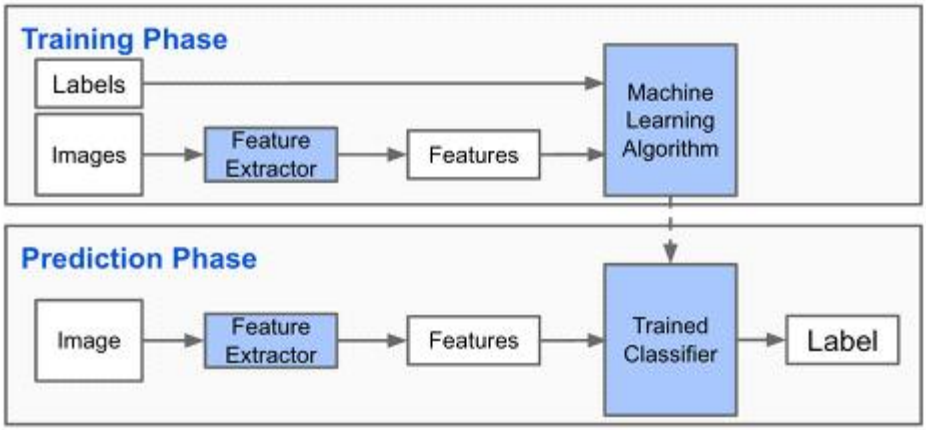

그림 5. 머신러닝 단계

머신 러닝을 통해서 해결할 수 있는 작업은 '패턴 인식, 이상 감지, 예측'등이 있다. 패턴 인식은 실제 장면에서 사물 인식, 사람 얼굴 혹은 얼굴의 감정 표현 인식, 음성 단어 인식 등이 있다. 이상 감지는 비정상적인 일련의 신용 카드 거래 감지, 원자력 발전소 센서 판독 값의 비정상적인 패턴 감지 등이 있다. 마지막으로 예측은 미래의 주식 가격 혹은 통화 환율 예측, 특정 인물이 좋아할 만한 영화 예측 등이 있다.

간단하게 예를 들자면 아래와 같은 일이다.

입력	일	출력
글자(텍스트)	문자 인식	정확도
사진	영역 식별	정확도
이메일	스팸 여부 판단	정확도

글자나 사진과 같은 경우는 단연 픽셀의 값이 들어올 것이다. 픽셀 정보를 기반으로 어떤 입력 데이터를 분석하여 머신 러닝 데이터 모델은 이의 여러 픽셀 정보의 학습에 따라 파라미터를 조정하여 학습의 효율(성능)을 높일 것이다.

이러한 머신러닝의 종류에는 4가지가 있다.
- 지도 학습 (Supervised Learning)
- 비지도 학습 (Unsupervised Learning)
- 반지도 학습 (Semi-supervised Learning)
- 강화 학습 (Reinforcement Learning)

그 중, 대표적으로 지도 학습과 비지도 학습이 있다.
지도 학습은 다른 말로 감독 학습이라고도 불리는데 일반적으로 입력 데이터와 출력 데이터를 나타내는 하나의 데이터 셋(Data Set)을 가지고 새로운 입력 데이터를 받았을 때 출력 데이터를 예측하는 방법이 기본 아이디어이다.
즉, 특징이 이미 정해진 데이터를 사용하여 학습하는 방법으로 각 데이터에 정해진 특징은 레이블(label)이라고도 표현할 수 있다. 레이블이 있는 데이터들의 집합은 트레이닝 세트(Training Set)라고도 부른다.

다음은 특정 사물에 해당하는 사진을 짝지어둔 데이터이다. 사진이라는 데이터에 해당하는 특징(사물)을 짝지어 둔 데이터들의 집합이므로 이는 트레이닝 세트의 한 예로 들 수 있다.

그림 6. 트레이닝 세트의 예시

위와 같이 입력 데이터(사진의 픽셀 정보)를 주고 '이 이미지는 개, 고양이다'라고 이미지에 레이블링을 한다.
(컴퓨터는 픽셀의 수치적인 정보만 받을 뿐 무엇인지 전혀 알지 못한다.)
우리는 Training Data Set의 정보를 주고 새로운 이미지를 넣었을 때 결국 개인지 고양이를 구별하는 어떤 모델을 만들고 싶을 것이다.
이와 같이 지도 학습에서 사용하는 트레이닝 데이터 셋의 정보는 문제와 답의 한 쌍으로 이루어진 데이터를 나타낸다.

이와 같은 트레이닝 세트를 통해 각 레이블에 대한 특징을 학습하게 된다.
즉, 그래프 상으로 표현했을 때 X축은 데이터의 특성을 나타내는 어떤 숫자의 나열 또는 다차원 배열의 형태이며 Y축은 그 데이터와 맵핑되는 레이블일 것이다.
트레이닝 데이터 세트는 다음과 같이 표를 사용하여 표현할 수도 있다.

X_1	X_2	X_3	Y
3	6	9	3
2	5	7	2
2	3	5	1

이 표에서 X는 자료의 특징을 표현할 때 사용하고, Y는 레이블을 의미한다.
이렇게 주어진 트레이닝 세트를 학습하게 되면 이 데이터들을 기반으로 한 모델이 생성되고, 이 모델을 사용하면 어떠한 특징을 갖는 데이터가 어떤 레이블에 속할지 추측할 수 있게 된다.

비지도 학습은 지도 학습과 달리 학습에 사용하는 데이터에 특징(레이블)이 부여되어 있지 않다. 지도 학습이 기존에 있는 데이터를 기반으로 새로운 데이터에 대한 특징을 추론하는 것을 목표로 한다면, 비지도 학습은 주어진 데이터들이 어떻게 구성되어 있는지를 분석하는 것을 목표로 한다.
비지도 학습의 대표적인 예로 구글 뉴스 서비스(비슷한 주제의 뉴스끼리 묶어줌)와 단어 클러스터링(유사한 단어끼리 묶어줌)이 있다.

지도 학습과 비지도 학습을 간단히 설명할 수 있는 영화가 있는데, 채피(Chappie, 2015)라는 영화이다.

그림 7. 머신러닝이 담겨 있는 영화 '채피'

영화에서 보면 주인공은 채피라는 인공지능 로봇에게 처음에 '이것은 시계야!',
'나는 ○○야!'라는 식으로 설명한다.
그것을 통해서 기계는 학습을 하게 되고, 이것을 지도 학습이라 할 수 있다.

그림 8. 지도 학습 예시 (영화 채피)

반면에 혼자 TV를 보고 만화에 나오는 주인공의 행동을 따라하면서 스스로 무언가를 한다. 이것이 비지도 학습(Unsupervised Learning)이다.

그림 9. 비지도 학습 예시 (영화 채피)

지도 학습은 종류에 따라 회귀분석과 분류로 나뉜다.
회귀분석은 어떠한 변수에 영향을 받는 결과가 연속적인 경우에 사용한다. 시험공부에 투자한 시간(변수)에 따라 예상되는 기말고사 점수(0~100 사이의 연속적인 값)을 추측하는 모델을 대표적인 예로 들 수 있다.
다음은 시험공부에 투자한 시간과 실제로 획득한 성적을 담고 있는 트레이닝 세트이다.

X (시험공부에 투자한 시간)	Y (점수)
10	90
9	80
3	50
2	30

앞의 트레이닝 세트로 학습시킨 회귀분석 모델을 사용한다면, 시험공부에 7시간을 투자한 학생의 예상 점수를 대략 75점 정도로 예측할 수 있다.

분류는 어떠한 변수에 영향을 받는 결과를 연속적이지 않은 값들로 나눌 때 사용한다.

시험공부에 투자한 시간(변수)에 따라 예상되는 합격 여부(Pass/Fail) 혹은 학점 (A/B/C/D/E/F)을 추측하는 모델을 대표적인 예로 들 수 있다.
다음은 시험공부에 투자한 시간과 실제로 획득한 학점(Pass/Fail)을 담고 있는 트레이닝 세트이다.

X (시험공부에 투자한 시간)	Y (Pass/Fail)
10	P
9	P
3	F
2	F

여기에서는 데이터를 합격(P) 혹은 불합격(F) 두 가지로 나뉜다. 따라서 이러한 데이터 구분은 Binary Classification이라 불린다. 합격 여부가 아닌 학점을 추측하는 모델을 만드는 경우, 다음과 같은 트레이닝 세트를 사용하게 될 것이다.

X (시험공부에 투자한 시간)	Y (학점)
10	A
9	B
3	D
2	F

주어진 데이터를 두 개 이상으로 분류하였으므로 이는 Multi-label classification이라 할 수 있다. 결국 지도 학습은 분류(classification)와 회귀 분석(regression analysis)을 활용한 것이다.

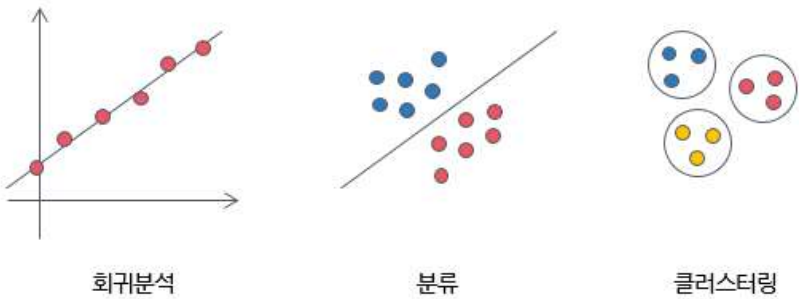

그림 10. 회귀분석, 분류, 클러스터링 표현도

2.1 인공지능 구현

2.1.2 딥러닝

딥러닝은 인공지능을 구성하기 위한 인공신경망(ANN, Artifical Neural Networks)에 기반 하여 컴퓨터에게 사람의 사고방식을 가르치는 방법이며, 사람이 가르치지 않아도 컴퓨터가 스스로 사람처럼 학습할 수 있는 인공지능 기술이다.

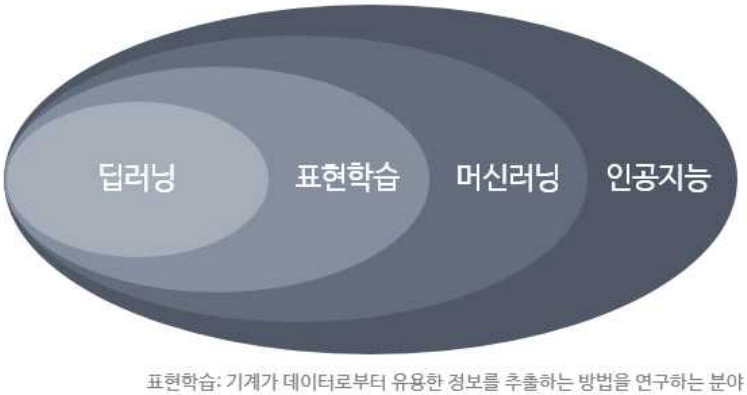

그림 11. 인공지능 기술의 집합

우선 사람의 학습이 어떻게 이뤄지는지 가장 작은 것부터 확인을 해야 한다.
사람의 뇌에는 뉴런이라는 수많은 신경 세포가 있다. 뉴런은 서로 전기신호를 주고받으며 정보를 교환하는데, 뉴런의 행동을 살펴보면 어떻게 이렇게 단순한 행동으로 이뤄질까 싶을 정도로 간단하다.

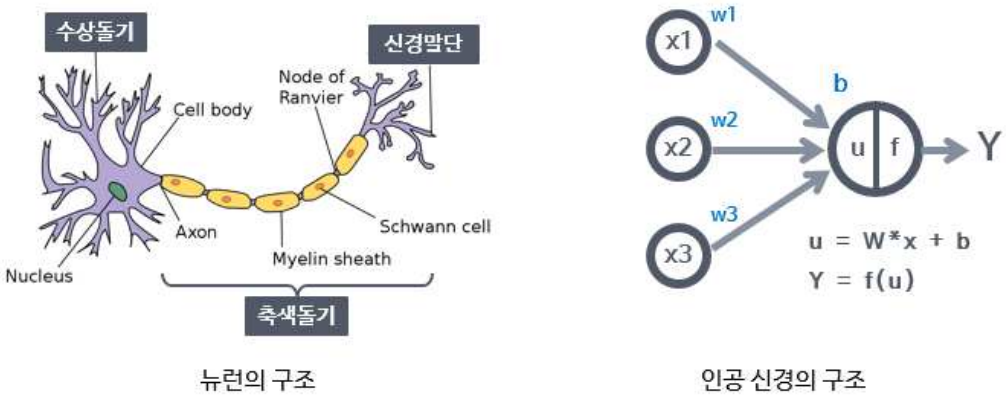

그림 12. 뉴런과 인공 신경의 구조

뉴런이 여러 수상돌기를 통해 전기신호를 받으면 각각 가중치를 곱하고 특정 값을 더한 후 축색돌기를 통해 이동한다. 그리고 그 값이 어떤 값 이상이라면 신경말단에서 다음 뉴런으로 넘어가고 아니면 넘어가지 않는다. 이 뉴런을 수학적으로 만든 인공신경의 구

조를 보면 input(x)이 들어오면 어떠한 가중치(W)가 곱해지고, input들의 합에 임계값(b)이 더해진 값이 u가 된다. 그 후에 f(u) 즉, 모든 값을 더한 결과(u)를 activation function 활성화 함수(f)에 넣어 계산을 한다. 그 값(Y)이 일정 값 이상이면 1이 되어 활성화가 되고, 아니면 0이 되어 활성화가 되지 않는다. 활성화 함수는 기준 범위 내에서의 값을 추출하게 하는 것이다.

딥러닝은 하나의 뉴런이 아닌 수천~수억 개의 뉴런들이 연결되어 이뤄진다. 즉, 인공 신경 또한 수많은 연결들을 통해 망을 이룬다.

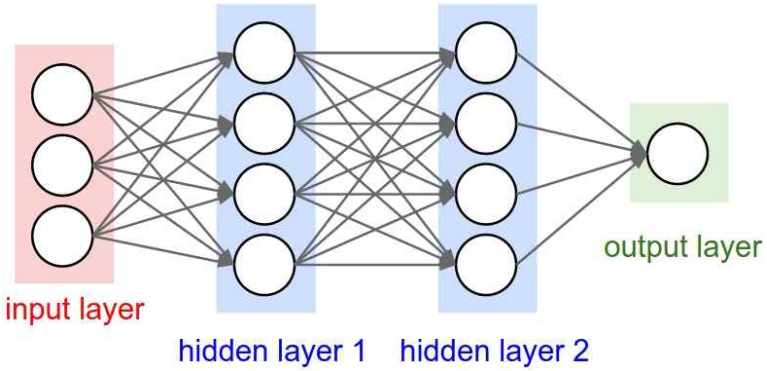

그림 13. 인공 신경망 구조

우리의 뇌에 셀 수 없이 많은 뉴런들이 서로 연결되어 하나의 망(network)를 이루고 있고, 우리 또한 이러한 인공 신경들을 묶어 인공 신경망을 구성해야 사람처럼 학습을 하게 할 수 있을 것이다. 크게 입력층(input layer)과 출력층(output layer) 그리고 그 사이에 있는 layer들을 묶어서 은닉층(hidden layer)이라고 한다. 위 그림에서는 모든 노드가 연결되어 있지만, 꼭 다 연결되어 있어야 하는 것은 아니다. 단지 어떻게 신경망을 구성할지에 따라 정해지는 것이다. 예를 들어, 딸기 사진을 입력에 넣었다면 출력에는 딸기라는 결과가 나와야 맞는 것이다. 딸기가 맞다면 그냥 넘어가면 되지만 딸기가 아니라고 결과가 나왔다면 이것은 문제가 있는 것이다. 따라서 다시 뒤로 돌아가 각 인공 신경의 가중치(W)와 임계값(b)을 수정하게 된다. 이러한 과정을 반복하며 W와 b값을 조정하고 딸기라는 결과가 나오도록 학습하는 것이다.

딥러닝에서 가장 중요한 알고리즘은 오차 역전파이다. 오차 역전파 알고리즘은 비용 함수의 값을 떨어뜨리면서 가중치 값을 학습한다. 결과 값을 통해서 다시 역으로 input 방향으로 오차를 다시 보내며 가중치를 재등록 하는 것이다. 물론 결과에 영향을 많이 미친 노드(뉴런)에 더 많은 오차를 돌려줄 것이다.

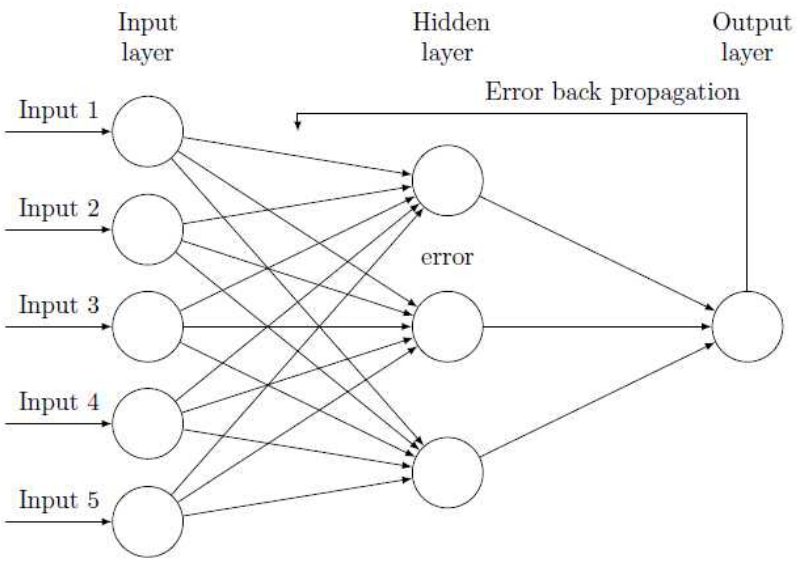

그림 14. 오차 역전파 표현도-1

위 그림을 보면 input이 들어오는 방향(순전파)으로 output layer에서 결과 값이 나온다. 결과 값은 오차(error)를 갖게 되는데 역전파는 이 오차(error)를 다시 역방향으로 hidden layer와 input layer로 오차를 다시 보내고 가중치를 계산하면서 output에서 발생했던 오차를 적용시키는 것이다. 한 번 돌리는 것은 1 epoch 주기라고 하며 epoch를 늘릴수록 가중지착 계속 학습되면서 점점 오차가 줄어나가는 방법이다.

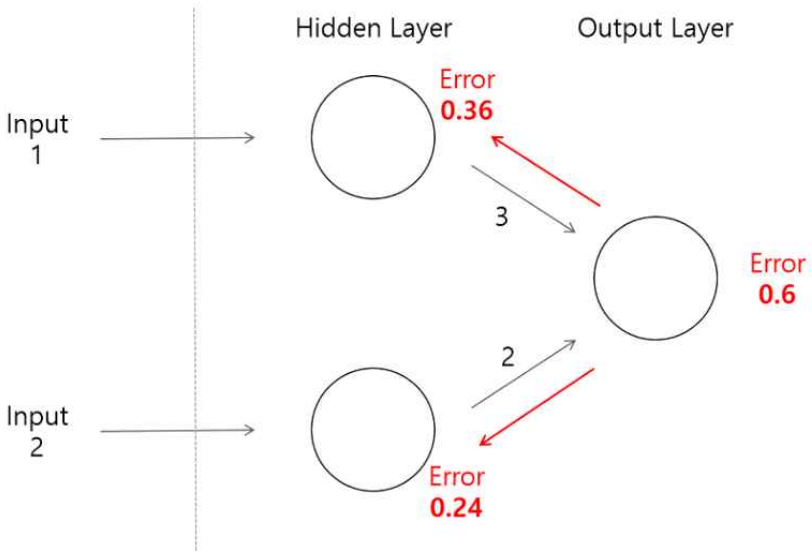

그림 15. 오차 역전파 표현도-2

위의 그림을 보면 Output layer에서 나온 결과 값이 가진 오차가 0.6이라고 되어 있다. 이전 노드(뉴런) Output layer에 각각 3, 2라는 값을 전달하였기 때문에 Output의 Error

에 위 노드는 60%, 아래 노드는 40% 영향을 주었다고 볼 수 있다. 균등하게 가중치를 나눠줄 수 있지만 영향을 미친 만큼 오차를 다시 역전파하게 된다.
error 0.6을 0.6, 0.4를 곱하니 위 노드에는 error가 0.36이, 아래 노드에는 0.24가 전달된다. 오차 역전파는 말 그대로 이렇게 오차를 점점 거슬러 올라가면서 다시 전파하는 것을 의미한다.

2.1 인공지능 구현

2.1.3 Tensorflow

Tensorflow는 계산을 그래프로 표현하는 프로그래밍 시스템이다. 여기서 계산이란 다른 말로 노드라고도 한다. 즉, 노드를 그래프로 표현하고 그 그래프를 실행하기 위한 과정이다. Tensorflow는 구글 브레인 팀에서 개발을 하였고 지금 머신러닝 분야에서 엄청나게 사용 중이다. 차세대 번역기 또한 Tensorflow를 사용하여 구현되었다.

Tensorflow에서 다루는 데이터의 단위는 Tensor(n차원 행렬)이다. 여러 차원의 배열의 형태를 한 숫자 값의 모음으로 구성되어 있다. Tensor의 rank는 차원의 수이다.
Tensorflow Core 프로그래밍은 아래의 두 단계로 생각할 수 있다.
- Computational graph(계산 그래프) 구성
- Computational graph(계산 그래프) 실행

계산 그래프는 일련의 Tensorflow 연산자를 노드 그래프로 배열한 것이다. 각 노드는 0개 이상의 Tensor를 입력 값으로 받아서 Tensor를 출력 값으로 생산한다. 노드의 한 유형은 상수이다. 모든 Tensorflow 상수와 같이 입력 값이 없고 내부적으로 저장하는 값을 출력한다. 그래프의 노드는 계산이라고도 불리지만 연산이라고도 불린다. 연산, 계산, 노드, 오퍼레이션 모두 같은 말이다. 노드에서는 각종 수학 계산, 데이터 입출력, 데이터의 읽기/저장 등을 수행한다. 하나의 노드는 0개 이상의 Tensor를 input으로 취할 수 있고, 취한 Tensor를 이용하여 어떤 계산을 한 뒤, 0개 이상의 Tensor를 output으로 다시 반환한다.
Tensorflow에서 '그래프(graph)'란 일종의 계산 방식을 기술한 설계도라고 할 수 있다. 어떤 것이든 계산하기 위해서는 반드시 그래프를 Session에 올려야 한다. 세션이란 그래프의 실행환경을 캡슐화한 것이다. 캡슐화된 실행환경을 객체화하기 위하여 Session객체를 만들어 그래프를 실행한다. 그래프를 실행하기 위해서는 반드시 Session 객체가 필요하다. Session은 그래프를 하나 혹은 여러 개의 CPU나 GPU 등의 device에 그래프를 올린 뒤, 연산을 실행할 수 있는 메소드를 제공한다. 이 메소드는 run()메소드를 말한다. 이 메소드는 연산에 의해 생성된 결과 텐서를 numpy의 ndarray객체의 형태로 반환한다.

2.1 인공지능 구현

2.1.4 Python과 Tensorflow를 활용한 인공지능 구현

차세대 번역기의 가장 기초는 손글씨 인식 인공지능 프로그램이다. 프로그래밍 언어로 Python3을 사용했고 행렬계산에 강력한 라이브러리인 numpy를 사용했다. 학습에 사용할 데이터로는 MNIST를 이용했으며 딥러닝을 공부할 때 반드시 거쳐야 할 'Hello, World' 같은 존재이다. MNIST는 60,000개의 트레이닝 셋과 10,000개의 테스트 셋으로 이루어져 있고 이중 트레이닝 셋을 학습데이터로 사용하고 테스트 셋을 신경망으로 검증하는 데에 사용한다.

그림 16. MNIST 데이터 그림 예시

학습 데이터는 아래 코드와 같다.

```
data_file = open("data/mnist_train.csv", "r")
training_data = data_file.readlines()
data_file.close()

test_data_file = open("data/mnist_test.csv", "r")
test_data = test_data_file.readlines()
test_data_file.close()
```

그림 17. MNIST 데이터 코드

readlines() 메서드는 파일의 내용을 한 줄씩 불러와서 문자열 리스트로 반환하는 함수이다. 데이터를 불러온 다음에 아래처럼 데이터를 확인해보았다.

```
print(training_data[0])

5,0,0,0
```

이제 해당 데이터를 이용하여 train 한다.

```
x = tf.placeholder(tf.float32, [None, 784])
W = tf.Variable(tf.zeros([784, 10]))
b = tf.Variable(tf.zeros([10]))

y = tf.nn.softmax(tf.matmul(x, W) + b)

y_ = tf.placeholder(tf.float32, [None, 10])
cross_entropy = -tf.reduce_sum(y_ * tf.log(y))
train_step = tf.train.GradientDescentOptimizer(0.01).minimize(cross_entropy)

init = tf.initialize_all_variables()
sess = tf.Session()
sess.run(init)

for i in range(1000):
    batch_xs, batch_ys = mnist.train.next_batch(100)
    sess.run(train_step, feed_dict={x: batch_xs, y_: batch_ys})
```

그림 20. MNIST train 코드

input은 총 55000개의 784 pixel을 가진 학습 데이터가 사용되며, output는 10개의 classification을 가진 55000개의 결과가 만들어질 것이다.

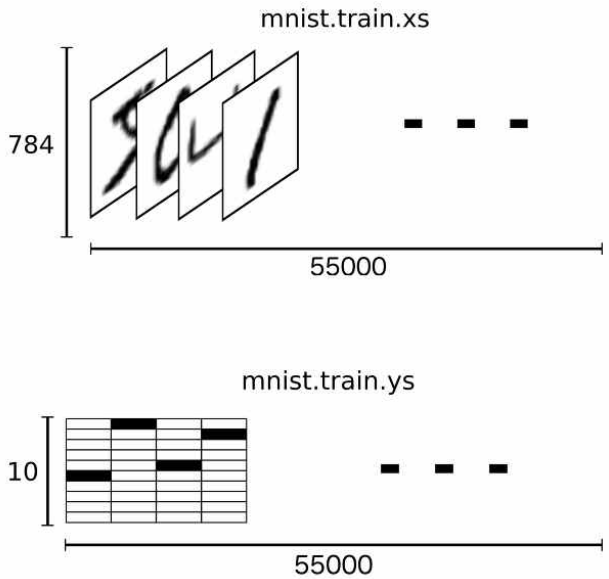

그림 21. MNIST train 표현도

이제 Tensorflow 연산 시 데이터를 Tensorflow에게 보내기 위한 공간을 만들 필요가 있다. W는 784 x 10 차원의 0 값을 가진 행렬로 정의했다. None x 784 행렬을 10개의 class로 분류하기 위해 W는 10차원의 행렬이 되어야 한다. 0은 초기 값이며 학습을 통

해 그 값을 계속 변경하였다. b도 10차원의 0 값을 가진 행렬로 정의해 두었다.
y값을 계산하는 것을 보면 tf.matmul(x,W) + b라고 적혀 있는데 이것은 단순하게 행렬 곱을 의미하며 방정식으로 말하자면 Wx + b를 의미한다. 이 결과 값에 대해 softmax라는 함수를 취하는데 해당 결과 값에 대해서 softmax를 취하게 되면 확률 값으로 변하게 된다.

8이란 필기체 사진(그림 16)을 예로 들어보면 위의 사진을 80% 가량 8이라고 인식할 수 있지만 10%가량은 9라고 인식할 수 있고, 그 나머지는 나머지 숫자들로 인식할 수가 있다. Wx는 해당 숫자가 무엇을 나타내는지 그 증거를 찾는 과정이라고 할 수가 있으며 b는 추가적인 증거를 더하는 과정이라고 생각하면 된다.
그 후에 loss 함수를 정의해 주는데, cross_entropy라 불리는 방식을 사용한다.
one_hot encode로 표현한 확률 분포와 실제 계산해서 나온 확률 분포 간의 차이를 구해서 그 값이 가장 작은 지점에서의 weight 값을 찾아내는 것이다.
loss 함수까지 정의가 끝났으면 이제 gradient descent optimizer에 learning_rate와 loss 함수를 등록해 주면 사전 작업은 모두 끝났다.

Training을 돌리기 전에 tf의 모든 변수들을 초기화 시켜준다. Tensorflow는 lazy evaluation 방식이라 코드가 작성되면 바로 실행되는 것이 아니라 session.run이 실행 되어야 실제 함수가 동작한다. 이제 for 문을 1000번을 돌려, 100개 씩 input_images 데이터와 input_labels 데이터를 가져온다. session.run을 실행하여 아까 정의한 training 함수를 실행시키면 모든 training이 완료된다.

```
correct_prediction = tf.equal(tf.argmax(y,1), tf.argmax(y_,1))
accuracy = tf.reduce_mean(tf.cast(correct_prediction, tf.float32))


print(sess.run(accuracy, feed_dict={x: mnist.test.images, y_: mnist.test.labels}))
```

그림 22. Training 데이터 테스트 코드

argmax는 해당 output 에서 가장 index가 큰 결과를 가져 온다. index가 크다는 의미는 가장 점수가 높게 설정되었다는 말이고 해당 결과를 정답으로 볼 수 있다는 말이 된다. 예측한 값에서의 argmax 와 실제 onehot encode에서의 argmax를 각각 가져와서 비교해 보고 해당 값이 같으면 true, 틀리면 false를 리턴할 것이다. correct_prediction은 true, false 배열을 나타낸다.
mnist.test.images와 mnist.test.labels의 실제 값들을 직접 보고 싶어 아래와 같이 작성한 후 확인하였다.

```
for i in range(1):
    batch_x, batch_y = mnist.test.next_batch(100)
    diff_a = sess.run(tf.argmax(y,1), feed_dict={x:batch_x})
    diff_b = sess.run(tf.argmax(y_,1), feed_dict={y_:batch_y})

    print diff_a
    print diff_b
```

```
[7 4 9 7 3 1 8 2 6 9 6 7 8 8 3 7 2 5 1 4 0 3 3 2 0 6 1 1 1 2 2 3 5 3 6 5 1
 7 0 3 0 1 5 0 5 9 4 2 3 4 2 6 2 1 1 2 0 6 5 9 6 1 6 9 0 7 0 0 2 1 5 5 7 1
 2 0 3 1 8 8 7 5 2 5 3 6 4 4 7 8 0 0 1 6 6 0 2 5 2 0]
[7 4 9 7 3 1 8 2 6 9 6 9 8 8 3 7 7 5 1 4 0 3 3 2 6 6 1 1 1 3 2 3 5 3 6 5 1
 7 0 3 0 1 5 0 5 3 4 2 3 4 2 6 2 1 1 2 0 4 5 9 6 1 6 9 0 7 0 0 0 1 5 5 7 1
 2 3 3 1 8 8 7 5 2 5 3 6 4 4 7 8 0 0 1 6 2 0 2 5 2 5]
```

그림 23. MNIST 테스트 코드와 결과

조금 더 편하게 볼 수 있도록 아래와 같이 수정하면 그림이 출력된다.

```
for i in range(2):
    result_boolean = []
    batch_x, batch_y = mnist.test.next_batch(9)
    diff_a = sess.run(tf.argmax(y,1), feed_dict={x:batch_x})
    diff_b = sess.run(tf.argmax(y_,1), feed_dict={y_:batch_y})
    print "sample output : " + str(diff_a)

    for k in range(9):
        if diff_a[k] == diff_b[k]:
            result_boolean.append("T")
        else:
            result_boolean.append("F")
    print "compare : " + str(result_boolean)

    plt.figure(i)
    coordi = [191, 192, 193, 194, 195, 196, 197, 198, 199]

    for index, image in enumerate(batch_x):
        image.shape(28,28)
        plt.subplot(coordi[index])
        plt.imshow(image)
print "sample input : "
```

```
total accuracy : 0.9147
sample output : [0 3 2 9 2 6 8 4 0]
compare['F', 'T', 'F', 'F', 'T', 'T', 'F', 'T', 'F']
sample output : [8 0 6 2 1 6 9 3 9]
compare['T', 'T', 'T', 'T', 'T', 'T', 'F', 'T', 'T']
sample input :
```

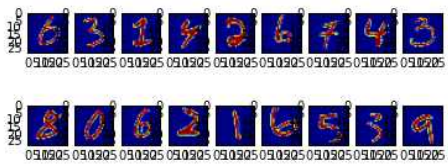

그림 24. MNIST 최종 결과

차세대 번역기의 원리이자 기초인 MNIST 코드가 모두 완성되었다.
이제 본격적으로 응용하여 다룰 다른 언어와 병행해 응용할 것이다.

2.2 프로그램 연동

2.2.1 소켓 통신

차세대 번역기는 여러 가지 플랫폼에서 작동할 수 있도록 개발되었다. Linux, Windows뿐만 아니라 macOS(Mac OS X), Android, iOS, 기타 Web Browsing이 가능한 모든 장치에서 사용이 가능하다. 차세대 번역기가 어떻게 모든 플랫폼에서 작동이 가능한지에 대해 설계에 대해 설명한다. 번역해조가 딥러닝을 위해 사용한 Tensorflow 라이브러리와 inception-v3 모델은 Linux에서의 동작이 원활하기 때문에 많은 플랫폼을 지원하기 위해 번역 프로그램 자체에 인공지능 기술을 탑재하지는 않고 소켓 통신(서버-클라이언트) 방식을 채택하여 개발하였다. 소켓 통신은 프로그램간의 정보를 교환하기 위한 통신 방법이다. 단일 프로그램 내부, 다중 프로그램 서로, 다른 시스템 사이에서도 서버와 클라이언트 환경을 구축하여 정보를 교환할 수 있다.
소켓 통신 방법으로 가장 많이 사용되는 프로토콜은 TCP방식과 UDP방식이 있다. TCP 방식은 신뢰성 연결로 3-way hand shake를 통해 데이터를 정확하게 보낼 수 있지만 데이터의 신뢰성을 보장하기 위해 속도가 UDP에 비해 느리지만 UDP방식은 비신뢰성 연결로 속도가 TCP보다 빠른 대신 데이터의 일부가 손실될 가능성이 있다.
번역해조의 차세대 번역기는 사진의 정확한 전송을 위해 TCP방식의 소켓 통신으로 개발하기로 하였다.

2.2 프로그램 연동

2.2.2 프로토콜

프로토콜이란 간단히 말해 통신을 하는데 있어 서버와 클라이언트가 어떤 모양의 데이터(패킷)를 송신/수신해야 하는지 정하는 규칙이다.

사진의 크기 4 bytes	사진의 이진코드(binary) 데이터 ??? bytes

그림 25. 이미지를 보내고 받는 패킷

사진의 크기는 각 사진 파일마다 크기가 다르기 때문에, 패킷의 길이는 항상 변동하게 된다. 그렇다면 이러한 패킷 유연하게 다룰 수 있어야하기 때문에 그림과 같이 패킷의 시작 4 byte는 사진의 크기(용량)를 최대 2^{32}(0~4,294,967,295)로 표현할 수 있는 범위로 결정하였다. 그 뒤는 사진의 이진 데이터가 나타난다.

10	사진의 이진코드(binary) 데이터 {??, ??, ??, ??, ??, ??, ??, ??, ??, ??}

그림 26. 이미지의 이진코드 데이터 변환

만약 10 byte 크기의 사진인 경우 그림과 같이 총 패킷의 길이는 14 byte가 된다. 수신측에서는 패킷의 첫 4 byte 만큼을 정수형으로 읽은 다음 사진의 크기가 10 byte라는 사실을 알 수 있으며, 앞으로 10 byte 만큼을 받은 다음, 다른 사진을 수신할 수 있도록 대기할 수 있어야 한다. 만약 사진의 크기를 알 수 없는 상태가 된다면 서버는 어느 정도 크기의 사진이 수신되는지 알 수 없어 분석 작업을 할 수 없으며 그것은 또 다시 다른 사진의 수신을 할 수 없는 상태이자 통신에서의 교착상태라고 볼 수 있다.

문장 ??? bytes

그림 27. 분석 결과(문장)를 보내고 받는 패킷

분석결과를 보내고 받는 패킷은 앞의 사진을 보내고 받는 패킷과 달리 간단하다. 문자로만 이루어진 데이터만 보내면 된다. 결과를 전송했다는 것은 차세대 번역기 프로그램의 분석 작업은 끝난 것으로 보고 더 이상의 소켓 통신의 연결을 필요로 하지 않아 연결을 종료하고 그 때까지의 패킷을 분석결과로 보는 것이다.

2.2 프로그램 연동

2.2.3 서버

서버는 항상 사진을 받을 수 있도록 해야 하기 때문에 차세대 번역기의 핵심 프로그램과 통합하여 구현되었다.

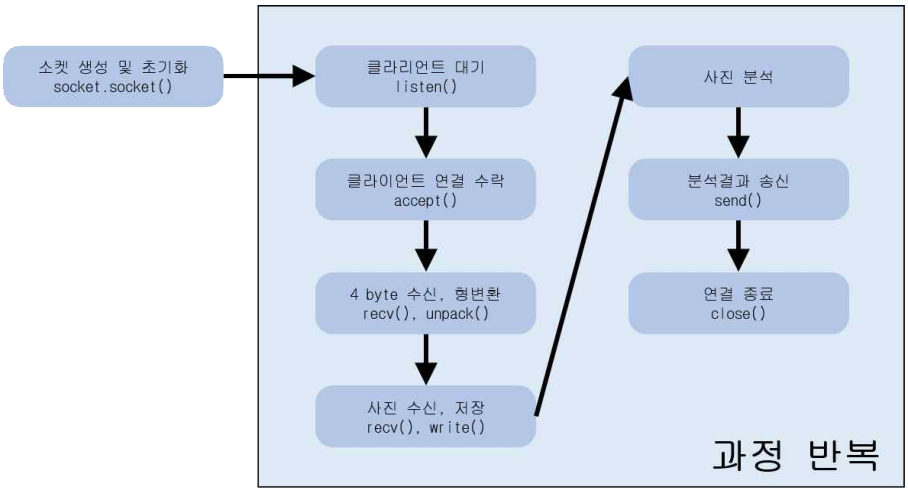

그림 28. 서버 구현 알고리즘

서버 프로그램이 처음 시작되면 소켓을 생성하고 초기화를 한다. 이 때, 포트번호는 RFC1700에 정해지지 않은(겹치지 않는) 임의의 포트 번호인 10001을 사용하였다.

```
HOST=''
PORT=10001

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT))
s.listen(1)
print('연결 대기중...')

conn, addr=s.accept()
print('연결 수락됨')
print(addr)
```

그림 29. 소켓의 초기화부터 클라이언트 연결까지의 Python 코드

서버(서비스 제공)의 소켓을 생성하고 클라이언트의 연결을 대기하다 클라이언트의 연결 요청이 있다면 연결을 수락하고 클라이언트와 데이터(패킷)를 주고받을 수 있는 상태가 된다.

```
data = conn.recv(4)
filesize = struct.unpack('<L', data)[0]
print( "받을 파일 사이즈: %d" % filesize )
```

그림 30. 소켓으로부터 읽은 데이터(값)를 대입

서버는 클라이언트와 연결이 된 직후 클라이언트가 패킷을 송신하는 경우 서버에서는 4 byte의 패킷을 받도록 설계되어 있으며 이 데이터를 32 bit Little-Endian 정수 형식으로 변환하여 받아야 할 사진 파일 크기를 filesize 변수에 기록한다.

```
filepath = IMAGE_ROOT + "recv.jpg"
f = file(filepath, 'wb')
recvsize = 0
while True:
  data = conn.recv(1024)
  f.write(data)
  recvsize += len(data)
  if recvsize >= filesize:
    break

f.close()
```

그림 31. 받는 사진 데이터를 저장

다음으로 실제 사진 파일의 데이터가 전송되는데 서버 소켓은 매 사이클마다 1,024 byte 씩 수신하여 data 변수에 임시로 기록하고, 디스크에 누적하여 저장한다. 또 이 data 변수의 크기를 항상 합산하여 앞서 기록된 사진 파일의 크기인 filesize 변수만큼 누적되는 경우 사진 파일의 수신이 완료되었음을 뜻하고 반복되는 Loop 문단을 나가고 파일 쓰기를 중지한다.

```
print("이미지를 분석합니다...")
try:
  result = main2(_, rFILE)
except:
  result = "*ERROR*"

conn.send(result)
conn.close()
s.close()
```

그림 32. 사진 속 결과를 클라이언트로 전송

직전 과정을 거쳐 저장된 사진 파일을 분석하여 문장을 도출해내고(main2 함수) 그 결과를 String 형식으로 클라이언트와 아직 연결된 소켓을 이용하여 클라이언트로 전송한 다음 연결을 종료함으로써 서버 구현이 종료된다.

2.2 프로그램 연동

2.2.4 클라이언트

Windows 클라이언트는 C#으로 개발되었으며 .NET Framework에서 제공하는 기본 소켓 통신 라이브러리를 사용한다.

```
string temp;
string host;
int port;

temp = ClassUtil.getIni("Server", "host");
if (temp.Length == 0) temp = "192.168.0.50";
host = temp;

temp = ClassUtil.getIni("Server", "port");
if (temp.Length == 0) temp = "10001";
port = int.Parse(temp);

this.Invoke(dS, "서버와 연결을 시도하는 중입니다...");

IPAddress ipAddress = IPAddress.Parse(host);
IPEndPoint remoteEP = new IPEndPoint(ipAddress, port);

Socket sender = new Socket(ipAddress.AddressFamily, SocketType.Stream, ProtocolType.Tcp);

string result;
result = "... Failed Translate ...";

try
{
    sender.Connect(remoteEP);
    this.Invoke(dS, "서버와 연결이 성공하였습니다...");
```

그림 33. C#에서 Python 서버로 소켓을 연결하는 모습

Windows 클라이언트는 결합도를 낮추기 위해 INI 파일에서 서버의 IP와 PORT정보를 불러오도록 설계하였다. 서버의 정보를 C# 소켓 통신에 맞는 객체를 생성하여 입력한 다음 소켓을 생성한 뒤 서버에 연결을 시도한다.

```
// 보내는 프로토콜:
// 파일용량(4바이트) + 파일데이터

// 이미지 불러오기
string filename = FormMain.getInstance().currentImagePath;
byte[] filedata = System.IO.File.ReadAllBytes(filename);
byte[] filesize = BitConverter.GetBytes(filedata.Length);
sender.Send(filesize);
sender.Send(filedata);

this.Invoke(dS, "이미지를 분석하고 있습니다...");

// 받는 프로토콜:
// 결과문자열(ascii)
byte[] bytes = new byte[1024];
int bytesRec = sender.Receive(bytes);
result = Encoding.ASCII.GetString(bytes, 0, bytesRec);

// 소켓 연결 해제
sender.Shutdown(SocketShutdown.Both);
sender.Close();
```

그림 34. 데이터를 Python 서버로 전송하고,
결과를 받아오는 모습

소켓의 연결이 성공한 다음은 즉시 사진 파일을 이진 형식으로 byte 배열에 적재한 다음 그 크기를 구한다. 그리고 프로토콜에 맞게 사진 파일 크기를 4 byte로 전송하고 이어서 사진 파일 데이터를 보낸다. filedata.Length는 배열 크기를 나타내는데 32 bit unsigned int 형식이자 4 byte로써 우리가 설계한 프로토콜에 맞는 크기이다.
전송이 끝나면 서버 측에서 사진을 분석하고, 결과를 되돌려주는데 이때 클라이언트는 최대 1024 byte 만큼 수신할 수 있는 상태이며 받는 데이터는 byte 배열이기 때문에 String으로 변환을 해준다.
마지막으로 소켓 연결을 해제함으로써 Windows 클라이언트 소켓 통신 부분이 종료된다.

모바일 단말기와 다른 플랫폼의 클라이언트에서 또한 차세대 번역기를 사용할 수 있도록 구현하였다. 간단하게 WEB과 연동되어 있다는 점인데 WEB 부분에서 이 내용을 다루도록 한다.
번역해조는 2018 컴퓨터공학과 졸업 작품 전시회에서 관람객에게 작품을 보여주는 것 뿐만이 아니라 체험을 해볼 수 있도록 방안을 모색하는 도중 WEB 페이지를 사용하여 체험 존을 개발해 관람객의 참여를 끌어낼 수 있겠다는 생각을 했다. 또한, 대부분의 단말기에서 WEB 브라우징을 할 수 있기 때문에 어떠한 플랫폼에 구속받지 않고 서비스를 제공할 수 있을 것이라 본다.

2.2 프로그램 연동

2.2.5 웹

번역해조는 2018 컴퓨터공학과 졸업작품 전시회에서 관람객에게 작품을 보여주는 것 뿐만이 아니라 체험을 해볼 수 있도록 방안을 모색하는 도중 WEB 페이지를 사용하여 체험존을 개발해 관람객의 참여를 끌어낼 수 있겠다는 생각을 했다. 또한 대부분의 단말기에서 WEB 브라우징을 할 수 있기 때문에 어떠한 플랫폼에 구속받지 않고 서비스를 제공할 수 있을 것이라 본다.

웹 서버는 차세대 번역기 핵심이 구동되는 서버에 함께 서비스된다.
번역해조는 Linux 환경에서 가장 사용하기 편리하도록 개발된 APM을 구하여 설치하였다. APM은 Apache web server, PHP, MySQL이 통합된 도구이다. Apache web server는 컴퓨터에서 웹 서버를 제작할 수 있도록 도와주는 프로그램이며, PHP는 SSS(Server side script)언어로 동적인 웹 페이지를 개발할 수 있게 만들어주는 웹 언어이다.
마지막으로 사용된 MySQL은 DBMS(데이터베이스 관리 시스템)의 한 종류인 프로그램이다.

웹 서버는 차세대 번역기 핵심이 구동되는 서버에 함께 서비스된다. 번역해조는 Linux 환경에서 가장 사용하기 편리하도록 개발된 APM을 구하여 설치하였다. APM은 Apache web server, PHP, MySQL이 통합된 도구이다. Apache web server는 컴퓨터에서 웹 서버를 제작할 수 있도록 도와주는 프로그램이며 PHP는 SSS(Server side script)언어로 동적인 웹 페이지를 개발할 수 있게 만들어주는 웹 언어이며 MySQL은 DBMS(데이터베이스 관리 시스템)의 한 종류인 프로그램이다.

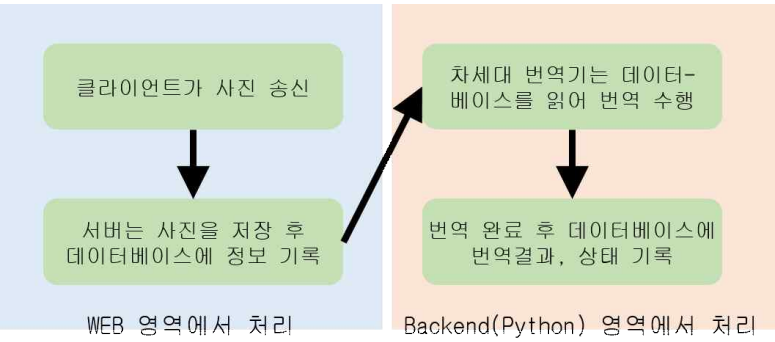

그림 35. 영역 처리 알고리즘

차세대 번역기 웹 서버의 동작 방식은 앞서 소켓 통신과 유사하다. 이미 웹 서버는 소켓 통신을 통해 웹 페이지(데이터)를 전송하므로 프로토콜의 설계 없이 동작의 설계만 있으면 된다.

```
$link = @mysqli_connect('localhost', 'root', 'root');
if (!$link) {
    echo "오류: 데이터베이스 연결에 실패하였습니다.";
    return;
}
@mysqli_select_db($link, "trial");

$filename = @$_FILES['filename']['name'];
$filesize = @$_FILES['filename']['size'];

if( strlen($filename) <= 0 ){
    echo "오류: 파일 이름이 이상합니다. ㅠㅡㅠ $filename";
    return;
}

$ext = pathinfo($filename, PATHINFO_EXTENSION);

mysqli_query($link, "INSERT INTO image (filename, result, state, regdate)
                     values ('-', '-', -1, NOW())");
$dbid = mysqli_insert_id($link);

$date = date("Y-m-d-H-i-s");
$newfilename = $date . "-" . $dbid . "." . $ext;

$tmp = $_FILES['filename']['tmp_name']; // 임시 파일

$info = getimagesize($tmp);
if($info === FALSE){
    echo "오류: 이미지 파일이 아닌 것 같습니다.";
    return;
}

if (($info[2] !== IMAGETYPE_GIF) && ($info[2] !== IMAGETYPE_JPEG) &&
   ($info[2] !== IMAGETYPE_PNG) && ($info[2] !== IMAGETYPE_BMP)) {
    echo "오류: 허가된 이미지 형식이 아닙니다.";
    return;
}
```

그림 36. PHP에서 MySQL에 연결하고
새로운 사진 정보를 삽입하는 모습

클라이언트가 사진을 업로드를 요청하는 경우 데이터베이스에 사진 파일의 이름, 번역 결과, 번역 상태를 나타내는 테이블에 번역 상태(state) 열을 업로드 중인 상태로 하나의 레코드를 삽입한다. 이 테이블은 기본키(Primary key)가 자동 증가 정수형으로 DBMS가 새로운 기본키를 발급하게 설계되어 있으며 레코드 삽입이 완료되면 현재 삽입된 레코드의 기본키에 해당하는 정수를 반환받아 dbid 변수에 기억하고 있는다. 다음으로 사진 파일의 이름을 중복되지 않게 정한 다음 사진 파일의 크기, 형식을 판단하여 업로드 및 저장 과정을 완료한다.

```
if(move_uploaded_file($tmp, $fullpath)){
    // png면 jpg로
    $newfilename = $date . "-" . $dbid . ".jpg";
    $image = @imagecreatefrompng($fullpath);
    @imagejpeg($image, "./upload/".$newfilename, 100);
    @imagedestroy($image);

    @mysqli_query($link, "UPDATE image SET filename = '$newfilename', state = 0 WHERE id = $dbid");
    echo "성공: $dbid";
} else {
    echo "오류: 업로드에 실패하였습니다.";
}
```

그림 37. 저장 코드

저장에 성공한다면 이전에 기록한 데이터베이스의 레코드에서 filename을 파일 이름으로, state는 0으로 수정한다.

```
def getImageFromDatabase():
  isFind = 0

  while( isFind == 0 ):
    conn = pymysql.connect(host='localhost', user='root', password='root', charset='utf8', db='translator')
    try:
      with conn.cursor() as cursor:
        sql = "SELECT id, filename, regdate FROM image WHERE state = 0 ORDER BY id ASC"
        cursor.execute(sql)
        result = cursor.fetchone()
        if result is not None:
          rID = result[0]
          rFILE = result[1].encode("utf-8", "replace")
          rREGDATE = result[2]
          isFind = 1
    finally:
      conn.close()
    time.sleep(1)
```

그림 38. PHP에서 업로드된 사진 정보를 Python이 불러오는 모습

Python으로 개발된 서버는 데이터베이스에서 state열의 값이 0인(=업로드된 사진이지만 번역되지 않은 사진) 하나의 행이 나올 때까지 1초마다 반복적으로 모니터링한다. 만약 state열의 값이 0인 레코드가 발견된다면 ID, 파일 이름, 등록 시간 열을 읽어 정보를 가져온다.

```
def saveResultToDatabase(rID, rRESULT):
  print("데이터베이스에 이미지 번역 결과를 기록합니다...")
  conn = pymysql.connect(host='localhost', user='root', password='root', charset='utf8', db='translator')
  try:
    with conn.cursor() as cursor:
      if rRESULT != "*ERROR*": # 오류가 아니면
        sql = "UPDATE image SET result = %s, state = 1 WHERE id = %s"
      else:
        sql = "UPDATE image SET result = %s, state = -2 WHERE id = %s"
      cursor.execute(sql, (rRESULT, rID))
      conn.commit()
  finally:
    conn.close()
  print("결과 저장 완료!!")
```

그림 39. Python이 번역 결과를 데이터베이스에 기록하는 모습

Python 서버는 소켓 통신과 같이 사진 분석 및 번역 작업을 수행하고 결과를 받는다. 결과를 받는데 성공했다면 다시 데이터베이스에 연결하여 이전 단계에서 불러왔던 기본키가 ID인 레코드의 정보를 번역 결과 문장과 번역 성공 상태를 나타내는 값을 기록하는 것으로 WEB 서버 작업이 종료된다.

2.3 UI 구현

2.3.1 개발 도구

어떠한 운영체제(이하 OS)에서의 구동을 기준으로 개발을 진행할지 선택하는 과정에서 가장 사용자들이 보편적으로 사용하게 되는 OS에서 구동이 가능하도록 진행하였다.
그 기준으로 선택된 OS는 마이크로소프트에서 개발한 윈도우이며, 글로벌 시장조사기관인 '넷애플리케이션'에 따르면 2015년 5월 기준, 전체 데스크톱 운영체제 시장에서 윈도우의 점유율은 91.07%에 이르기에 윈도우를 채택하게 되었다.
윈도우 어플리케이션 개발에는 여러 종류가 있지만 이중 가장 보편적인 C++ MFC 방식과 C# Winform 두 가지의 방식을 결정 테이블위에 올려 프로젝트 개발에 가장 알맞은 방식을 채택하였다.
채택 기준으로는 시간대비 개발의 효율성이 높은 객체 지향적 프레임워크를 사용하는지가 선택의 중요 기준이 되었다.
Winform방식은 .NET Framework에서 Microsoft Windows 응용 프로그램을 만들기 위한 것으로 이 프레임워크는 현대적이고 개체 지향적이며 확장 가능한 클래스 집합을 제공하므로 Windows 기반의 강력한 응용 프로그램을 개발할 수 있다.
그렇기에 Winform방식이 프로젝트 진행에 효율성이 높을 것으로 예상되기에 Winform방식을 채택하였다. 이를 이용하기 위한 개발 도구로는 Visual Studio 2017 을 선택하였다.

2.3 UI 구현

2.3.2 Visual Studio

마이크로소프트에서 개발한 통합 개발 환경으로 Visual Studio 하나만으로 윈도우에서 돌아가는 일반 적인 어플리케이션은 모두 만들 수 있다.
공식적으로 Winform 개발을 지원하며 강력한 디버깅 기능과 다양한 라이브러리들을 지원하기에 윈도우 개발에는 탁월한 개발 환경으로 판단되어 채택하였다.
또한 대학생을 위한 무료 버전을 지원하는 부분이 가장 큰 장점 중 하나였다.
UI를 구성할 때 드래그 앤 드롭을 지원한다. UI부분을 코드로 작성하게 되는 것이 아닌, 단순한 드래그 앤 드롭 작업으로 컴포넌트를 배치한 다음, 배치 된 컴포넌트에 ID만을 설정해주면 코드 상에서 바로 쓸 수 있는 굉장히 빠르고 강력한 UI제작 기능을 지원한다.
또한 res를 알아서 관리해주기에 컴포넌트의 삭제, 수정, 추가가 굉장히 용이하다.

그림 40. Winform으로 제작한 차세대 번역기

2.3 UI 구현

2.3.3 안드로이드

스마트폰의 보급으로 인하여 사람들은 언제 어디서나 컴퓨터를 사용할 수 있으며 인터넷에 접속 할 수 있는 시대가 도래했다.

그렇기에 사람의 삶에 도움을 줄 수 있는 번역 프로그램 역시 모바일기기에서 작동할 수 있게 하하는 것이 공학이라는 인간의 삶의 질을 향상시킨다는 궁극적인 목적에 부합하는 것으로 생각하고 안드로이드 운연체제에 부합하는 안드로이드 어플리케이션을 개발하였다.

모바일 개발을 위해서는 안드로이드 스튜디오를 윈도우에 설치하여 개발을 진행하였다.

안드로이드 스튜디오에서의 UI개발은 XML을 이용하여 개발이 이루어지면 마치 웹 프로그래밍에서 HTML을 이용한 디자인처럼 사용자의 입맛에 맞는 유동적인 디자인이 가능하다.

XML은 W3C에서 개발된, 다른 특수한 목적을 갖는 마크업 언어를 만드는데 사용하도록 권장하는 다목적 마크업 언어이다. XML은 SGML의 단순화된 부분집합으로, 다른 많은 종류의 데이터를 기술하는 데 사용할 수 있다. XML은 주로 다른 종류의 시스템, 특히 인터넷에 연결된 시스템끼리 데이터를 쉽게 주고받을 수 있게 하여 HTML의 한계를 극복할 목적으로 만들어졌다.

XML은 문서를 사람과, 기계 모두가 읽을 수 있는 형식을 갖도록 규정하고 있으며 W3C가 XML 설계 목표에서 단순성과 일반성, 그리고 인터넷을 통해 사용 가능성을 강조하고 개발을 하였기에 유니코드를 사용하여 전 세계 언어를 지원한다.

이렇기에 XML을 이용한 디자인 방식은 굉장히 단순하고 스마트폰이라는 인터넷과 가까운 기기에서 어쩌면 완벽할 수 있는 디자인 언어이다.

비록 XML로 모든 UI배치부터 디자인을 구현 할 수 있다지만 안드로이드 스튜디오는 드래그 앤 드롭을 방식을 지원하기에 더 빠르고 정확한 UI개발이 가능하도록 한다. 코드 작성으로 UI배치를 진행 할 경우 정확히 배치되고 있는지 실시간으로 파악하기가 힘들며

구도가 상상되지 않기에 많은 시간이 소요된다. 이것을 안드로이드 스튜디오는 컴포넌트들을 마우스 드래그 앤 드롭으로 원하는 위치에 빠르게 배치가 가능하기에 개발의 편의성을 굉장히 높여준다.

3. 결론

3.1 활용 사례

인공지능의 대표적인 사례로는 구글 딥 마인드(Google DeepMind)가 개발한 바둑 프로그램 알파고와 암을 진단하는 IBM사의 닥터왓슨이 있다. IBM ‘왓슨’은 현재 가장 인공지능에 가까운 플랫폼 중 하나이다. 닥터 왓슨은 우리 지역 조선대학교병원에서도 도입하여 운영하고 있다. 인간과의 퀴즈, 번역 대결뿐만 아니라 암 진단까지 하는 다양하고 전문적인 인공지능이 전 세계인들을 놀라게 하고 있고, 하루가 다르게 다양한 산업군에서 인공지능이 개발되고 있는 상황이다.

키바(KIVA)는 아마존 글로벌 물류센터에 도입된 로봇으로 최대 1.4t까지 짐을 들어 옮길 수 있다. 오토드로우(AutoDraw)는 구글에서 개발한 로봇으로 그림 실력 없어도 사용자가 대충 그린 그림을 인식하여 디자이너와 일러스트레이터들이 그린 데이터를 검색, 유사한 이미지를 추천하는 인공지능 서비스이다. 딥페이스(DeepFace)는 사용자 자신의 얼굴이 있는 단체사진 한 장을 인식시키면, 그 사진에 포함된 페이스북 유저의 인적 네트워크를 분석하는 인공지능 서비스이다. 소프트뱅크의 페퍼(Pepper)는 일본의 세계최초 휴머노이드 로봇으로 사람의 감정과 주변상황을 파악하여 자율적인 판단이 가능한 로봇으로 다양한 기업의 고객접수, 도우미, 간병역할을 하고 있다. 국내 LG유플러스, 우리은행, 교보문고, 가천대 길병원, 롯데백화점, 이마트 등 기업에 도입되어 손님들에게 이용안내, 예약서비스를 돕고 있다니 기대가 된다. 채팅하는 로봇으로는 챗봇(Chatbot) 이 있다. 이는 인터넷 메신저 서비스와 인공지능의 만남으로 별도의 앱을 실행하지 않고도 기업용 메신저에 채팅하듯 질문하면 인공지능이 마치 사람처럼 대화하며 대답하는 서비스이다. 과거 '심심이'가 사용자의 질문을 단순 자동응답 해주는 서비스가 있었지만 현재는 대화형 플랫폼으로 발전하면서 주목받고 있다. 페이스북, 위챗, 구글 등 글로벌 업체에서 챗봇을 이미 도입하고 있고, 국내 기업에서도 챗봇을 활용하여 24시간 고객의 질문에 응대하고 있다. 그밖에도 다양한 분야에서 규모에 따라 인공지능을 통해 새로운 가치를 만들 기회가 많다.

이미지에서 사물을 인식하고자 할 때 사물의 종류를 1만 개라면 1만 가지의 출력단을 가지는 분류 모델을 구축하는 것이다. 페이스북의 ‘딥페이스’는 인간 수준으로 얼굴을 인식할 수 있고, 스탠포드대학 비전랩은 사진 속 사물을 문자로 표현해주는 ‘덴스캡’을 개발했다. 더 나아가 구글에서는 사진 속 장면 인식과 자동 번역 기술을 융합하여 완전한 문장으로 묘사해주는 알고리즘을 발표하기도 했다. 이 때 장면 인식에는 CNN 기술, 언어로 표현하는 단계에서는 RNN 기술을 이용되었다.

출처: Justin Johnson et al, 'Fully Convolutional Localization Networks for Dense Captioning', CVPR, 2016

그림 41. 덴스캡(DenseCap)의 사진 해석 결과

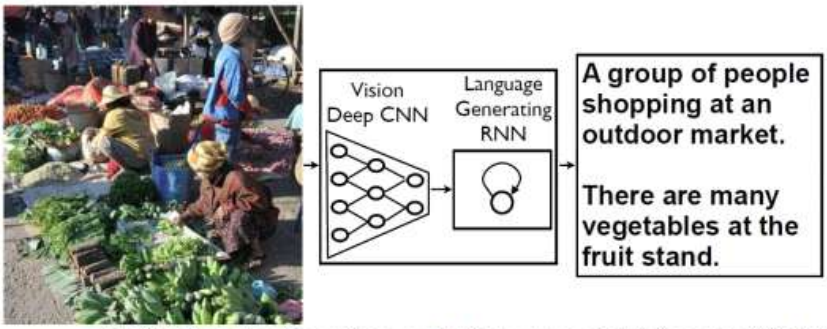

그림 42. 구글의 사진 속 장면 인식 및 문장 묘사 기술

반 고흐, 피카소 등 유명 화가의 화풍을 어떻게 특징화하여 요약할 수 있을까? 이것 역시 인간의 논리와 언어로 표현할 수 없는 대단히 추상화된 영역의 일이다.

딥러닝은 추상화된 특징표현을 스스로 발견할 수 있어서, 학습을 통해 반 고흐의 작품인지 아닌지를 순식간에 판단 할 수 있다. 과거에는 사람의 표정이나 목소리에서 감정을 읽어 내거나, 입술의 움직임만으로 대화를 해석하는 것은 정신적이고 추상적인 일이어서 인간만이 할 수 있다고 믿었다. 그러나 이 또한 Emotient와 LipNet 등의 사례와 같이 딥러닝을 통해 구현되고 있다.

그림이나 이미지 외에도 음성, 동영상과 동작 등으로부터 각종 패턴을 인식하는 인공지능은 여러 분야에서 시도되고 있다. 의료 분야를 예로 들자면 미국의 스타트업 MedyMatch는 의료영상 기반 뇌졸중 진단 AI 기술을 개발 중이고, 스탠포드대학 연구진이 웨어러블 기기를 이용해 사람의 질병을 미리 예측하는 기술을 공개하였다. 하버드대 연구진은 인스타그램에 올린 사진의 채도, 색상, 자신의 표정 등을 분석하여 우울증을 70% 적중률로 예측하는 알고리즘을 개발하였다.

그림 43. 의료기술에 적용된 딥러닝

콘텐츠 업계에서는 어떤 콘텐츠를 소비자가 좋아할 것인가를 분류/예측함으로써 맞춤형 추천을 제공할 수 있다. 미국의 판도라(Pandora)와 넷플릭스(Netflix)는 각각 음악과 VOD을 추천하기 위해 인공지능 기술을 활용하고 있다. 단순히 콘텐츠를 추천하는 것을 넘어 집안의 가전제품을 제어하거나 날씨와 뉴스 정보를 읽어주는 등 종합 인공지능 에이전트 서비스도 속속 등장하고 있다. 대표적인 서비스로 아마존의 'Echo', 그리고 kt의 'GiGAGenie', SKT의 '누구' 등이 있다.

그림 44. 콘텐츠 업계에 적용된 딥러닝

그 외에는 문맥과 상황에 맞게 자연스러운 번역을 가능케 하는 기계 번역, 챗봇 알고리즘 등에서도 딥러닝 기술을 활용할 수 있다. 그리고 사물인식, 동작인식 등 다양한 모델의 집약체로 볼 수 있는 자율주행 자동차 분야에서도 상용화를 위한 연구가 활발하다. 자율주행 자동차 분야에는 테슬라(Tesla), 구글(Google)을 비롯하여 포드, GM, 재규어 등의 기존 메이저 자동차 제조사들도 관련 기술의 스타트업을 인수하며 경쟁에 뛰어들었다.

그림 45. 자동차 기술에 적용된 딥러닝

인공지능의 창작 활동 딥러닝으로 학습한 유명 화가의 화풍을 분류나 예측이 아닌 다른 용도로 활용한 흥미로운 시도가 있었다. 명화의 특징표현 정보를 합성하여 다른 사진을 그 화가의 화풍으로 변환시키는 것이다. 이 기술을 이용하면 이미 세상을 떠난 거장의 그림을 무한대로 창조해 낼 수 있다. 인공지능이 예술작품을 만들어 낼 수 있다는 것이다. 구글은 이 방법을 활용하여 입력한 사진을 다소 몽환적인 느낌으로 변환시키는 '딥드림'을 공개하였고, 그 작품을 모아 전시회까지 열었다. 트위터의 '딥포저'와 러시아 개발사가 공개한 'Prisma'도 사용자가 사진을 올리면 피카소, 렘브란트 등 유명 화가의 화풍으로 변형시켜주는 유사한 서비스를 제공하고 있다.

그림 46. 사진을 유명 화가의 화풍으로 변환한 예

나날이 발전하고 있는 인공지능 사례를 보니 4차 산업혁명 시대, 기계의 지능화를 실감할 수 있다. 그리고 인공지능의 '데이터'와 사람의 '창의성'이 융합되어 앞으로 어떤 놀라운 서비스가 실현될지 기대가 된다. 하지만 기존에 사람이 했던 노동력을 대체하기 때문에 4차 산업혁명 시대의 미래의 일자리에 대해선 꼭 고민이 필요할 것으로 보인다.

3.2 후기

[강준호]

인공지능 기술은 오래전부터 많이 들어왔지만 익숙하진 않았습니다. 어렸을 때부터 많은 공상 과학 기술 영화들을 보며 언젠가 인공지능이 사람을 대체하기 시작하는 시대가 올 것이고, 더 나아가 인공지능이 사람을 지배하는 시대가 올 것이라고 생각했습니다. 옛날부터 사람들은 인공지능이 활약하는 시대는 미래시대라고 생각하였습니다.

하지만, 핸드폰마다 존재하는 인공지능 비서, 알파고와 같은 다양한 인공지능 기술들은 실제로 우리의 삶에 밀접하게 다가왔습니다.

인공지능은 더 이상 미래의 기술이 아닌 현재의 기술입니다. 4차 산업 혁명과 더불어 인공지능기술의 중요성이 대두되고 있는 시기에 딥러닝을 통하여 다양한 인공지능기술에 접하여보고, "차세대 번역기"라는 좋은 주제를 통해 직접 구현하고 훈련시키며 인공지능 기술과 한층 가까워 질 수 있는 좋은 기회를 가지게 되었습니다.

결과물뿐만 아니라 구현하는 과정에 있었던 시행착오조차 모두 너무나 유익한 시간이었습니다.

생소한 개념이지만 미래를 이끌어 갈 수 핵심 개념이라 할 수 있는 인공지능이, 더 많은 경험의 장이 생겨 다양한 사람들이 쉽게 접할 수 있는 기회가 될 수 있도록 차세대 번역기가 도울 수 있었으면 좋겠습니다.

[남궁재우]

아이들에게 영어교육을 할 때 단어도 중요하지만 가장 이해하기 편한 설명은 '상황에 대한 설명' 이라고 생각했습니다.

영어 교육을 어떻게 하면 더 쉽게 할 수 있을지 고민하던 도중 새로운 번역기를 구상하게 되었고, 팀원들도 이에 적극 동의하였습니다.

하지만 딥러닝으로 구현해야 되므로 어떻게 접근해야할지 막막했었습니다. 그런 저희를 도와주기 위해 노흥식 지도교수님께서 방중에 인공지능 수업을 해주셨습니다.

텐서플로우부터 Python까지 모든 것이 처음이였지만 팀원들과 함께 노력하니 어느새 프로그래밍이 완성되었습니다.

4명이 각자 주특기인 언어를 사용해 각자 맡은 일에 최선을 다했습니다.

저는 기본 인공지능 조사를 하면서 프로그램의 기본 뼈대를 다졌습니다.

지금까지 배웠던 것과 완전히 다른 내용이라 오류도 많이 나고 시간도 오래 소모되었지만 팀원들과 함께 동거동락하면서 힘을 내었습니다.

본 프로그램이 아직 상용화 되어 있지는 않지만 만약 실제로 사용된다면 영어교육뿐만 아니라 여행을 위해 사용되는 번역기의 업그레이드 버전에 적용되었으면 좋겠습니다.

여기까지 올 수 있었던 강준호, 신우섭, 오준석 친구들에게 감사의 마음을 전합니다.

[신우섭]

4년간 대학교에서 배워온 모든 것의 결실을 맺는 졸업 잠품을 준비하면서 정말 많은 상황이 있었습니다. 개발 도중 막히는 부분이 있다면 4년이란 기간은 생각보다 길지 않았구나, 좀 더 열심히 공부하여 더 많은 것을 배워둘 것을 하고 후회도 해보았습니다.

내가 가진 지식이 아직은 많이 부족하다는 것을 깨달았습니다. 하지만 4년이라는 기간이 헛되지만은 않았습니다. 수업을 들으며 배웠던 것들이 졸업 작품에서 모두 사용되었으며 수업 중에 이것이 정말 실무에 도움이 될까 의심하였던 부분들도 실제로 사용되었습니다. 오히려 그 부분들을 더 자세히 공부하지 않았기에 개발의 완성도가 낮았던 부분도 있습니다.

하지만 아직 배움의 시간이 끝났다고 생각하지 않기에 앞으로 학교를 벗어나 사회에 발을 디뎌도 항상 학구열을 잃지 않으며 계속해서 배움을 실천할 것입니다.

그리하여 졸업 작품으로 끝이 아닌 앞으로 접할 많은 프로젝트에서 졸업 작품을 계기로 얻은 깨달음들을 실천할 것입니다.

[오준석]

첫 번째로 힘들었습니다. 팀 단위로 졸업 작품을 설계함에 있어 각자 맡은 부분을 개발하는 도중 서로 다른 의견이 있기도 하였고 이미 임의로 개발을 진행하여 각 모듈간의 프로토콜이 일치하지 않아 통신이 되지 않아 어느 한 쪽이 반대에 맞게 수정하거나 서로 모두 수정하는 일이 있었습니다. 두 번째로 좋은 경험이었습니다. 힘들었던 부분이 있었기 때문에 앞으로 팀 단위의 프로젝트를 설계할 때 의견을 모아야 할 부분이나 어떻게 개발할지 어느 것을 먼저 개발할지에 대한 지식이 많이 쌓였기 때문입니다. 아무래도 졸업 작품은 정말로 컴퓨터공학 학사가 되기 전 이러한 경험과 깨달음을 주기 위한 것이라 생각됩니다. 세 번째로 완벽했습니다. 졸업 작품을 완성하고 전시회를 성공적으로 마치고 난 후에도 소프트웨어공학적으로 우리 팀의 작품을 평가해본 결과 각 모듈과 함수들이 결합도가 낮게 설계되었다는 점을 볼 수 있었습니다. GUI 또한 어떠한 사용자더라도 멘탈 모델에 아주 가깝게 디자인되었다는 점도 볼 수 있었습니다. 끝으로 우리 팀은 졸업 이후에도 아주 좋은 개발자들이 될 수 있다고 생각합니다. 처음 졸업 작품 개발에 돌입하기 전에는 아주 막막했었는데 성공해냈기 때문입니다.